

Workflow: Generación de Datos para Nuevas Moléculas

Pipeline completo desde SMILES nuevos hasta datasets concatenados
V7 (HSQC 1 canal) y V8 (HSQC 2 canales con integración)

Lucas Passaglia

UCA Team

Abril 2026

Contents

1	Visión General del Pipeline	2
2	Paso 0 — Premisa: qué se tiene y qué se necesita	3
3	Paso 1 — Preparar los Archivos de Entrada de las 58k	4
4	Paso 2 — Generar los Vectores Objetivo (labels)	5
5	Paso 3 — Generar los Mapas HSQC	5
6	Paso 4 — Concatenar los Datasets	6
7	Paso 5 — Verificación de Consistencia	11
8	Paso 6 — Actualizar los Configs de Entrenamiento	13
9	Resumen: Lista de Verificación Completa	14
10	Consideración Adicional: el Split de Validación	14

1 Visión General del Pipeline

El objetivo es procesar 58.000 SMILES nuevos para ampliar la base de datos de 130k a $\sim 202k$ moléculas, generando **dos variantes de dataset** que alimentan arquitecturas distintas:

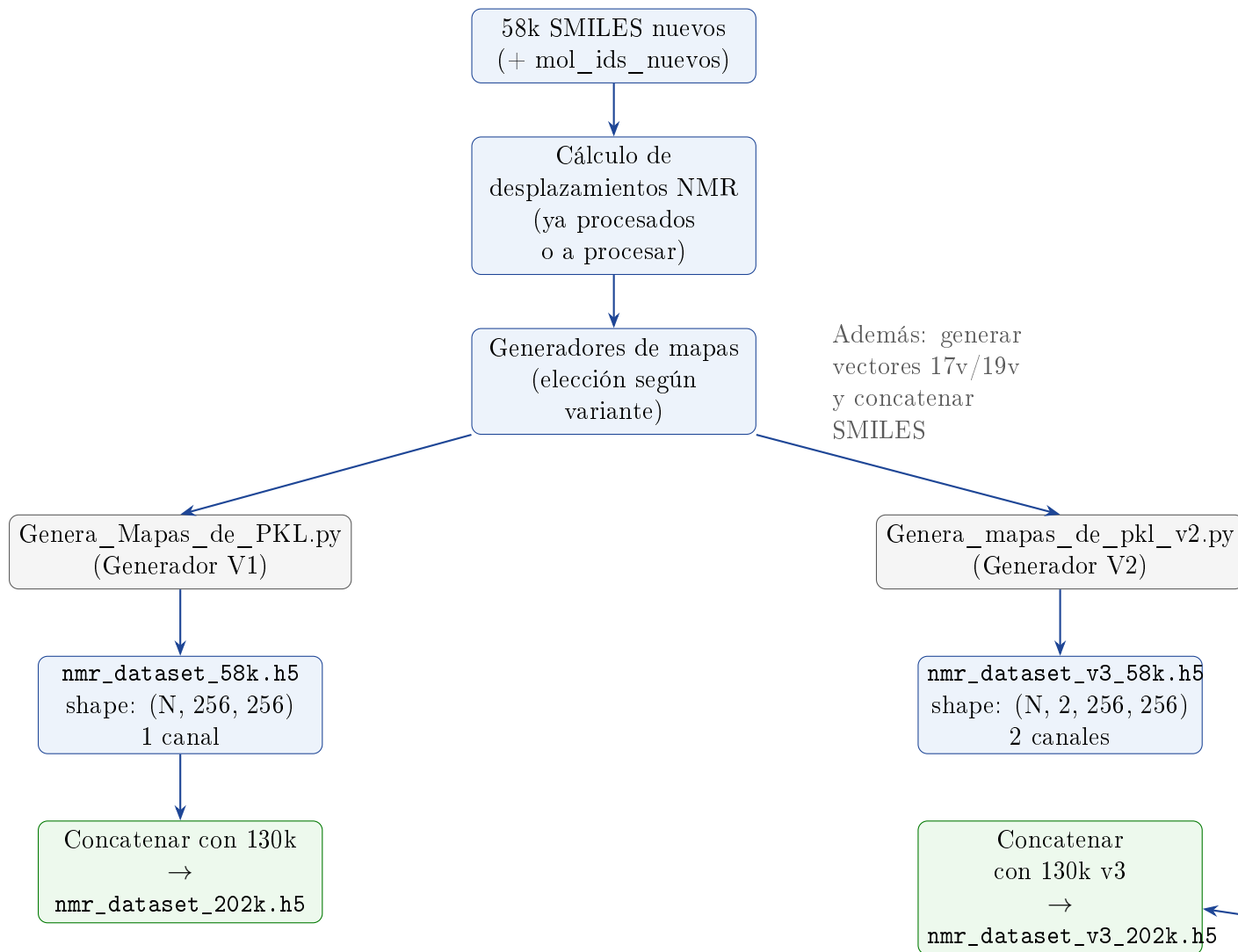


Table 1: Resumen de las dos variantes de dataset

Aspecto	Dataset V1 (para V7/V9)	Dataset V3 (para V8/V10)
Generador	<code>Genera_Mapas_de_PKL.py</code>	<code>Genera_mapas_de_pkl_v2.py</code>
HSQC shape	$(N, 256, 256)$ — 1 canal	$(N, 2, 256, 256)$ — 2 canales
Canal 0	DEPT fase ± 1 fija	DEPT escalado por N_H
Canal 1	No existe	Tipo CH: 0.33 / 0.67 / 1.0
vec_h	Amplitud uniforme (1 por H)	Amplitud $\propto N_H$
vec_c	Binning uniforme 256 bins	Binning no uniforme (128+96+32)
Nombre archivo	<code>nmr_dataset_{N}.h5</code>	<code>nmr_dataset_v3_{N}.h5</code>

2 Paso 0 — Premisa: qué se tiene y qué se necesita

Inventario de archivos existentes (130k)

Para los 130k originales ya existía:

- `smiles_144280.npy` — SMILES de las 144k moléculas originales
- `mol_ids_144280.npy` — IDs internos del PKL
- `nmr_calculated_data_scaled_144K.pkl` — desplazamientos DFT
- `nmr_dataset_144280.h5` — mapas generados con Generador V1
- `nmr_dataset_v3_144280.h5` — mapas generados con Generador V2
- `vectors_13c_17v_144280.npy` — vectores objetivo 17 clases
- `smiles_144280.npy` (ya mencionado)

Qué se necesita generar para las 58k nuevas

- `smiles_58k.npy` — SMILES de las moléculas nuevas
- `mol_ids_58k.npy` — IDs correspondientes en el PKL
- Los desplazamientos DFT para esas 58k (si ya están en el PKL, solo se extraen)
- `nmr_dataset_58k.h5` — mapas V1 (1 canal)
- `nmr_dataset_v3_58k.h5` — mapas V2 (2 canales)
- `vectors_13c_17v_58k.npy` — vectores objetivo 17 clases
- Opcionalmente: `vectors_13c_19v_58k.npy` para V9

Punto crítico antes de empezar: verificar que los 58k SMILES nuevos **no se solapan** con los 144k existentes. Si un SMILES aparece en ambos conjuntos, el modelo verá esa molécula tanto en entrenamiento como en validación, reintroduciendo el problema de fuga de datos que ya se corrigió. Hacer un `set()` de los SMILES y comparar antes de cualquier procesamiento.

3 Paso 1 — Preparar los Archivos de Entrada de las 58k

1a — Generar smiles_58k.npy y mol_ids_58k.npy

Si los 58k SMILES vienen de un archivo nuevo (por ejemplo, un CSV o texto plano), hay que convertirlos al mismo formato que los originales:

```
import numpy as np

# Cargar los SMILES nuevos (ajustar segun el formato de origen)
with open("nuevos_smiles.txt") as f:
    smiles_nuevos = [line.strip() for line in f if line.strip()]

# Generar IDs (deben coincidir con las claves del PKL)
# Si los IDs ya vienen del PKL, cargarlos del mismo archivo fuente
mol_ids_nuevos = [...] # lista de strings que son claves en el PKL

# Verificar solapamiento con los 144k originales
smiles_orig = set(np.load("smiles_144280.npy", allow_pickle=True))
solapamiento = [s for s in smiles_nuevos if s in smiles_orig]
print(f"Solapamiento: {len(solapamiento)} moléculas -- debe ser 0")

# Guardar
np.save("smiles_58k.npy", np.array(smiles_nuevos))
np.save("mol_ids_58k.npy", np.array(mol_ids_nuevos))
print(f"Guardadas {len(smiles_nuevos)} moléculas nuevas")
```

Los generadores de mapas buscan los archivos por el patron smiles_{N}.npy y mol_ids_{N}.npy donde N es el número de moléculas. Con 58k, los archivos quedarán como smiles_58000.npy y mol_ids_58000.npy. Ajustar el nombre al número exacto de moléculas que resulte.

4 Paso 2 — Generar los Vectores Objetivo (labels)

2 — Ejecutar Gen_vector.py sobre las 58k moléculas

El script `Gen_vector.py` que ya existe genera los vectores de grupos funcionales a partir de los SMILES. Debe correrse sobre los 58k nuevos apuntando al archivo correcto:

```
# En Gen_vector.py, cambiar la configuracion:
SMILES_FILE = "smiles_58k.npy"
OUTPUT_FILE = "vectors_13c_17v_58k.npy" # para V7/V8/V9
# OUTPUT_FILE = "vectors_13c_19v_58k.npy" # para V9/V10 con C-2X/C-3X

# Ejecutar normalmente
```

Resultado: `vectors_13c_17v_58k.npy` de shape (58000, 17).

Usar exactamente la misma definición de clases que en los 144k originales. Si `Gen_vector.py` fue modificado para 19 clases (C-2X y C-3X), generar dos archivos: uno de 17v y otro de 19v. No mezclar versiones al concatenar.

5 Paso 3 — Generar los Mapas HSQC

3a — Mapas V1 (1 canal) para modelos V7/V9

Configurar `Genera_Mapas_de_PKL.py`:

```
# Cambiar solo estas dos lineas en la configuracion:
PKL_PATH = r'ruta/al/nmr_calculated_data_scaled_144K.pkl'
WORK_DIR = r'ruta/donde/estan/smiles_58k.npy'

# El script detecta automaticamente smiles_58k.npy y mol_ids_58k.npy
# y genera: nmr_dataset_58000.h5 (o el N exacto de moleculas)
```

Resultado: `nmr_dataset_58k.h5` con datasets:

- `hsqc`: shape (N, 256, 256) float32
- `vec_c`: shape (N, 256) float32
- `vec_h`: shape (N, 256) float32

3b — Mapas V2 (2 canales) para modelos V8/V10

Configurar `Genera_mapas_de_pk1_v2.py`:

```
# Mismos cambios:  
PKL_PATH = r'ruta/al/nmr_calculated_data_scaled_144K.pkl'  
WORK_DIR = r'ruta/donde/estan/smiles_58k.npy'
```

```
# Genera: nmr_dataset_v3_58000.h5
```

Resultado: `nmr_dataset_v3_58k.h5` con:

- `hsqc`: shape (N, 2, 256, 256) float32
- `vec_c`: shape (N, 256) float32 (binning no uniforme)
- `vec_h`: shape (N, 256) float32 (amplitud proporcional)

Ambos generadores tienen **soporte nativo de resume**: si el job del clúster se interrumpe, al relanzarlo con el mismo archivo de salida detecta cuántas moléculas ya fueron guardadas y continúa desde ahí. No hace falta ningún cambio para esto.

6 Paso 4 — Concatenar los Datasets

Esta es la parte más delicada del pipeline. La concatenación debe ser **idéntica en orden** entre todos los archivos (h5, labels, smiles) para que cada fila i de la matriz de espectros corresponda exactamente al vector i de labels y al SMILES i .

4a — Concatenar los h5 (espectros)

```
import h5py
import numpy as np

def concatenar_h5(path_orig, path_nuevo, path_salida):
    """
    Concatena dos archivos HDF5 con las mismas claves (hsqc, vec_c, vec_h).
    Funciona para ambas variantes (1 canal o 2 canales): detecta el shape
    automáticamente.
    """
    with h5py.File(path_orig, 'r') as f_orig, \
        h5py.File(path_nuevo, 'r') as f_nuevo, \
        h5py.File(path_salida, 'w') as f_out:

        for key in ['hsqc', 'vec_c', 'vec_h']:
            data_orig = f_orig[key][:] # cargar todo en memoria
            data_nuevo = f_nuevo[key][:] # (aceptable para <50GB total)

            data_concat = np.concatenate([data_orig, data_nuevo], axis=0)

            # Crear dataset con compresion
            f_out.create_dataset(
                key,
                data=data_concat,
                compression='lzf',
                chunks=True
            )
            print(f" {key}: {data_orig.shape} + {data_nuevo.shape}"
                  f" -> {data_concat.shape}")

        print(f"Guardado: {path_salida}")

# --- Para V1 (modelos V7/V9) ---
concatenar_h5(
    path_orig = "nmr_dataset_144280.h5",
    path_nuevo = "nmr_dataset_58000.h5",
    path_salida = "nmr_dataset_202280.h5"
)

# --- Para V2 (modelos V8/V10) ---
concatenar_h5(
    path_orig = "nmr_dataset_v3_144280.h5",
    path_nuevo = "nmr_dataset_v3_58000.h5",
    path_salida = "nmr_dataset_v3_202280.h5"
)
```

Atención con la memoria: si los archivos h5 son muy grandes (el de 144k en V2 con 2 canales puede ser ~20 GB), cargar todo en RAM con [:] puede ser un problema. En ese caso usar el enfoque de copia por bloques:

```
# Alternativa para archivos muy grandes: copia en bloques
BLOQUE = 1000 # moléculas por bloque

with h5py.File(path_orig, 'r') as f_orig, \
    h5py.File(path_nuevo, 'r') as f_nuevo, \
    h5py.File(path_salida, 'w') as f_out:

    n_orig = f_orig['hsqc'].shape[0]
    n_nuevo = f_nuevo['hsqc'].shape[0]
    n_total = n_orig + n_nuevo

    for key in ['hsqc', 'vec_c', 'vec_h']:
        shape_final = (n_total,) + f_orig[key].shape[1:]
        dset = f_out.create_dataset(key, shape=shape_final,
                                    dtype='float32', compression='lzf',
                                    chunks=True)

        # Copiar orig en bloques
        for i in range(0, n_orig, BLOQUE):
            dset[i:i+BLOQUE] = f_orig[key][i:i+BLOQUE]

        # Copiar nuevo en bloques
        for i in range(0, n_nuevo, BLOQUE):
            dset[n_orig+i:n_orig+i+BLOQUE] = f_nuevo[key][i:i+BLOQUE]
```

4b — Concatenar labels (vectores objetivo)

```
import numpy as np

# Para 17 clases (V7/V8/V9)
labels_orig = np.load("vectors_13c_17v_144280.npy") # (144280, 17)
labels_nuevo = np.load("vectors_13c_17v_58000.npy") # (58000, 17)
labels_concat = np.concatenate([labels_orig, labels_nuevo], axis=0)
np.save("vectors_13c_17v_202280.npy", labels_concat)
print(f"Labels: {labels_orig.shape} + {labels_nuevo.shape}"
      f" -> {labels_concat.shape}")

# Opcionalmente para 19 clases (V9/V10) - mismo procedimiento
```


4c — Concatenar SMILES

```
smiles_orig = np.load("smiles_144280.npy", allow_pickle=True)
smiles_nuevo = np.load("smiles_58000.npy", allow_pickle=True)
smiles_concat = np.concatenate([smiles_orig, smiles_nuevo], axis=0)
np.save("smiles_202280.npy", smiles_concat)
print(f"SMILES: {len(smiles_orig)} + {len(smiles_nuevo)}"
      f" -> {len(smiles_concat)}")
```


7 Paso 5 — Verificación de Consistencia

5 — Script de verificación antes de lanzar entrenamiento

```
import h5py
import numpy as np

def verificar_dataset(h5_path, labels_path, smiles_path, nombre):
    print(f"\n=== Verificando {nombre} ===")
    with h5py.File(h5_path, 'r') as f:
        n_h5 = f['hsqc'].shape[0]
        shape_h = f['hsqc'].shape
        shape_c = f['vec_c'].shape
        shape_vh = f['vec_h'].shape
    n_labels = len(np.load(labels_path))
    n_smiles = len(np.load(smiles_path, allow_pickle=True))

    print(f"  hsqc shape : {shape_h}")
    print(f"  vec_c shape: {shape_c}")
    print(f"  vec_h shape: {shape_vh}")
    print(f"  N h5       : {n_h5}")
    print(f"  N labels   : {n_labels}")
    print(f"  N smiles   : {n_smiles}")

    ok = (n_h5 == n_labels == n_smiles)
    print(f"  Consistencia: {'OK' if ok else 'ERROR - NUMEROS NO COINCIDEN'}")

    # Verificar que no haya NaN o Inf en una muestra
    with h5py.File(h5_path, 'r') as f:
        sample = f['hsqc'][0]
        print(f"  NaN en muestra 0: {np.isnan(sample).any()}")
        print(f"  Inf en muestra 0: {np.isinf(sample).any()}")
        print(f"  Rango canal 0   : [{sample.min():.3f}, {sample.max():.3f}]")
        if sample.ndim == 3: # 2 canales
            print(f"  Rango canal 1   : [{sample[1].min():.3f}, {sample[1].max():.3f}]")
    return ok

# Verificar los cuatro archivos finales
verificar_dataset(
    "nmr_dataset_202280.h5",
    "vectors_13c_17v_202280.npy",
    "smiles_202280.npy",
    "Dataset V1 202k (para V7/V9)"
)

verificar_dataset(
    "nmr_dataset_v3_202280.h5",
    "vectors_13c_17v_202280.npy", # mismo labels
    "smiles_202280.npy",         # mismo smiles
    "Dataset V2/V3 202k (para V8/V10)"
)
```


8 Paso 6 — Actualizar los Configs de Entrenamiento

6 — Config YAML para V9 (scheduler corregido + 202k)

```
# config_v9_202k.yaml
experiment_name: "nmr_202k_v9"

paths:
  base_dir: "/home/lpassaglia.iquir/DB_202K"
  h5_filename: "nmr_dataset_202280.h5"
  labels_filename: "vectors_13c_19v_202280.npy" # 19 clases
  checkpoint_dir: "checkpoints_V9_202k"

hyperparameters:
  batch_size: 64
  learning_rate: 0.001
  epochs: 100
  val_split: 0.1
  optimizer: "Adam"
  scheduler:
    patience: 8 # scheduler corregido
    factor: 0.7

system:
  num_workers: 1
  pin_memory: True
  device: "cuda"

# config_v8b_202k.yaml (V8 + FM)
experiment_name: "nmr_202k_v8b"

paths:
  base_dir: "/home/lpassaglia.iquir/DB_202K"
  h5_filename: "nmr_dataset_v3_202280.h5" # 2 canales
  labels_filename: "vectors_13c_17v_202280.npy"
  checkpoint_dir: "checkpoints_V8b_202k"

hyperparameters:
  batch_size: 64
  learning_rate: 0.001
  epochs: 100
  val_split: 0.1
  optimizer: "Adam"
  scheduler:
    patience: 8
    factor: 0.7

system:
  num_workers: 1
  pin_memory: True
  device: "cuda"
```

9 Resumen: Lista de Verificación Completa

Table 2: Checklist del pipeline completo

Paso	Tarea	Herramienta	Listo
0	Verificar solapamiento de SMILES (debe ser 0)	Python set()	☐
1	Generar smiles_58k.npy y mol_ids_58k.npy	manual/script	☐
2	Generar vectors_13c_17v_58k.npy	Gen_vector.py	☐
2b	Generar vectors_13c_19v_58k.npy (para V9/V10)	Gen_vector.py	☐
3a	Generar nmr_dataset_58k.h5 (V1, 1 canal)	Genera_Mapas V1	☐
3b	Generar nmr_dataset_v3_58k.h5 (V2, 2 canales)	Genera_Mapas V2	☐
4a	Concatenar h5 V1: 144k + 58k → 202k	script concat	☐
4b	Concatenar h5 V2: 144k + 58k → 202k	script concat	☐
4c	Concatenar labels 17v: 144k + 58k → 202k	numpy concat	☐
4d	Concatenar labels 19v (si aplica)	numpy concat	☐
4e	Concatenar SMILES: 144k + 58k → 202k	numpy concat	☐
5	Verificar consistencia de los 4 datasets finales	verify script	☐
6	Actualizar configs YAML para V9 y V8b	editor	☐

Orden de prioridad si los recursos del clúster son limitados:

1. Paso 0 (verificación anti-leak): corre en segundos, no puede saltarse.
2. Pasos 1–2: rápidos, en CPU local.
3. Pasos 3a y 3b: los más lentos (~30–60 minutos cada uno). Lanzar en paralelo si hay dos nodos disponibles.
4. Pasos 4–6: minutos. Solo después de que 3a y 3b terminen correctamente.

10 Consideración Adicional: el Split de Validación

Decisión importante sobre el split de validación con el dataset ampliado.

Actualmente el set de validación de 14.428 moléculas es una partición del dataset de 144k con semilla 42. Cuando se concatene el dataset a 202k, hay dos opciones:

Opción A (recomendada): mantener la misma semilla y el mismo porcentaje (10%). El nuevo split dará ~20.228 moléculas de validación. Las 14.428 originales seguirán siendo validación, más ~5.800 nuevas. Los resultados serán comparables históricamente aunque no idénticos.

Opción B: congelar las 14.428 moléculas originales como set de validación permanente y usar todas las 58k nuevas solo para entrenamiento. Esto maximiza la comparabilidad histórica y garantiza que el EMA siempre se mide sobre las mismas moléculas. Requiere modificar el split en el dataset para excluir los índices originales de validación.

Para los propósitos del proyecto (comparar EMA entre versiones), la Opción A es más simple y suficientemente comparable. La Opción B es más rigurosa si el objetivo es publicar los resultados.

